

Relatório do Projeto

Processamento de Linguagens

Construção de um Compilador para Fortran 77 Standard

Grupo 36:

- A106936 - Duarte Escairo Brandão Reis Silva
- A106932 - Luís António Peixoto Soares
- A106856 - Tiago Silva Figueiredo

Departamento de Informática

Engenharia Informática 2025/2026

17 Maio 2026

Introdução

O presente relatório foi elaborado no âmbito da unidade curricular de Processamento de Linguagens. O trabalho desenvolvido enquadra-se num projeto de grupo que visa a construção de um Compilador para a linguagem de programação Fortran 77 Standard.

O objetivo foi conseguir aplicar na prática os vários conceitos e técnicas aprendidos ao longo do semestre, nomeadamente a análise léxica, sintática e semântica, bem como a tradução do código para a linguagem da máquina [EWVM](#).

O compilador desenvolvido foi escrito em *Python*, e para as análises léxica e sintática foi utilizada a biblioteca *PLY*.

Lexer

A análise léxica é dividida em dois módulos distintos: o **pré-processador** ([preprocessor.py](#)) e o **lexer** propriamente dito ([lexer.py](#)). Esta separação foi deliberada para respeitar o formato fixo do Fortran 77, onde a estrutura de colunas tem semântica posicional.

Pré-processador

O Fortran 77 usa linhas de 72 colunas com as seguintes zonas fixas:

Colunas	Índices Python	Significado
1	[0]	C ou * → linha de comentário
1–5	[0:5]	Label numérico (opcional)
6	[5]	Carácter de continuação (non-space, non-0)
7–72	[6:72]	Código Fortran executável

O pré-processador processa o código linha a linha e produz uma lista de **LogicalLine** (label, conteúdo, linha de origem). Quando encontra um carácter de continuação na coluna 6, concatena o código ao conteúdo da linha lógica em curso, permitindo que uma instrução se estenda por múltiplas linhas físicas. Toda a conversão para maiúsculas é feita aqui, pois o Fortran 77 é case-insensitive, evitando comparações nas fases seguintes. Erros de label inválido (não numérico) ou linha de continuação sem antecessora são reportados imediatamente como **PreprocessorError**.

Analizador léxico

O lexer usa **ply.lex** como motor de reconhecimento de padrões. As regras são funções Python com regex, ordenadas internamente pelo PLY por comprimento decrescente para evitar ambiguidades. Para cada **LogicalLine**, o PLY tokeniza apenas a zona de código; o label e o terminador de linha são injetados como tokens sintéticos construídos manualmente:

```
def _tokenize_line(self, ll: LogicalLine) -> list[lex.LexToken]:
    result = []
    if ll.label is not None:
        result.append(self._make_token(type_='LABEL', value=int(ll.label),
lineno=ll.src_line))
    self._lexer.input(ll.content)
    for tok in self._lexer:
        result.append(tok)
    result.append(self._make_token(type_='NEWLINE', value='\n',
lineno=ll.src_line))
    return result
```

O token **NEWLINE** é essencial pois o Fortran 77 é sensível à estrutura de linhas — é usado como terminador explícito em todas as produções gramaticais. Os identificadores são primeiro tokenizados como **IDEN** e depois reclassificados como keyword se pertencerem ao **frozenset** de palavras reservadas (O(1) de verificação). Os literais reais suportam notação científica com **E** e **D** (precisão dupla), e as strings aceitam aspas simples escapadas (' '). Os operadores lógicos e relacionais do Fortran (**.EQ.**, **.AND.**, etc.) têm regras próprias com maior prioridade que **IDEN**. Os erros léxicos são acumulados em lista (sem lançar exceção) e propagados no final para o parser.

Os tokens produzidos pertencem às seguintes categorias:

Categoria	Exemplos
Estrutura	PROGRAM , END , FUNCTION , SUBROUTINE , RETURN , STOP
Tipos	INTEGER , REAL , LOGICAL , CHARACTER
Controlo de fluxo	IF , THEN , ELSE , ENDIF , DO , CONTINUE , GOTO
I/O e intrínsecas	READ , PRINT , MOD , SQRT , MAX , MIN
Literais	INT_LIT , REAL_LIT , STRING_LIT , TRUE , FALSE
Operadores	OP_EQ , OP_NE , OP_LT , OP_LE , OP_GT , OP_GE , OP_AND , OP_OR , OP_NOT , POWER
Sintéticos	LABEL , NEWLINE , IDEN

Parser

O parser usa `ply.yacc`, que gera um autômato LALR(1). A gramática implementada cobre toda a estrutura do Fortran 77 suportado e é apresentada integralmente abaixo:

```

program          → program_unit
                  | program program_unit

program_unit     → main_program
                  | function_subprogram
                  | subroutine_subprogram

main_program     → PROGRAM IDEN NEWLINE body END NEWLINE

function_subprogram → type_spec FUNCTION IDEN '(' param_list ')' NEWLINE
body END NEWLINE
                  | FUNCTION IDEN '(' param_list ')' NEWLINE body END
NEWLINE
                  | type_spec FUNCTION IDEN '(' ')' NEWLINE body END
NEWLINE
                  | FUNCTION IDEN '(' ')' NEWLINE body END NEWLINE

subroutine_subprogram → SUBROUTINE IDEN '(' param_list ')' NEWLINE body END
NEWLINE
                  | SUBROUTINE IDEN '(' ')' NEWLINE body END NEWLINE

param_list       → IDEN
                  | param_list ',' IDEN

body             → decl_section stmt_section

decl_section     → ε
                  | decl_section declaration NEWLINE

stmt_section     → ε
                  | stmt_section stmt_line

stmt_line        → statement NEWLINE
                  | LABEL statement NEWLINE
                  | LABEL CONTINUE NEWLINE

declaration      → type_spec var_decl_list

type_spec        → INTEGER | REAL | LOGICAL | CHARACTER
                  | CHARACTER '*' INT_LIT

var_decl_list    → var_decl
                  | var_decl_list ',' var_decl

var_decl         → IDEN
                  | IDEN '(' INT_LIT ')'

```

```

statement          → assignment_stmt | if_stmt | do_stmt | goto_stmt
                    | print_stmt | read_stmt | call_stmt | return_stmt |
                    stop_stmt

assignment_stmt     → variable '=' expr

if_stmt             → if_then_block | logical_if_stmt

if_then_block       → IF '(' expr ')' THEN NEWLINE stmt_section ENDIF
                    | IF '(' expr ')' THEN NEWLINE stmt_section ELSE
NEWLINE stmt_section ENDIF

logical_if_stmt     → IF '(' expr ')' statement

do_stmt             → DO INT_LIT IDEN '=' expr ',' expr NEWLINE
stmt_section LABEL CONTINUE
                    | DO INT_LIT IDEN '=' expr ',' expr ',' expr NEWLINE
stmt_section LABEL CONTINUE

goto_stmt           → GOTO INT_LIT
print_stmt          → PRINT '*' ',' io_list
read_stmt           → READ '*' ',' io_list

io_list             → io_item | io_list ',' io_item
io_item             → expr

call_stmt           → CALL IDEN '(' expr_list ')' | CALL IDEN '(' ')'
return_stmt         → RETURN
stop_stmt           → STOP

expr                → expr arithmetic_op expr | expr logical_op expr
                    | expr relational_op expr
                    | unary_arithmetic_op expr | unary_logical_op expr
                    | '(' expr ')'
                    | INT_LIT | REAL_LIT | STRING_LIT | TRUE | FALSE
                    | variable | func_call

arithmetic_op       → '+' | '-' | '*' | '/' | POWER
logical_op           → OP_AND | OP_OR
relational_op        → OP_EQ | OP_NE | OP_LT | OP_LE | OP_GT | OP_GE
unary_arithmetic_op → '+' | '-'
unary_logical_op     → OP_NOT

variable            → IDEN
                    | IDEN '(' expr_list ')'
                    | IDEN '(' ')'

expr_list            → expr | expr_list ',' expr

func_call            → MOD '(' expr ',' expr ')'
                    | SQRT '(' expr ')'

```

```
| MAX '(' expr_list ')'  
| MIN '(' expr_list ')'
```

Precedência e associatividade

As expressões têm precedência resolvida pela tabela PLY (da menor para a maior):

Nível	Operadores	Associatividade
1 (menor)	.OR.	esquerda
2	.AND.	esquerda
3	.NOT.	direita (unário)
4	.EQ. .NE. .LT. .LE. .GT. .GE.	não-associativo
5	+ -	esquerda
6	* /	esquerda
7	UMINUS UPLUS	direita (unário)
8 (maior)	**	direita

Foram necessários dois pseudotokens (**UMINUS/UPLUS**) para distinguir os operadores unários dos binários **+/-**, e **DO_TERM** com a maior precedência para forçar a redução correta do terminador **LABEL CONTINUE** do DO loop em vez de o tratar como instrução do body.

Árvore Sintática Abstracta (AST)

Todos os nós são **dataclass** Python que herdam de **Node** (que guarda **lineno**). O método **accept()** implementa o padrão **Visitor por reflexão**, procurando o método **visit_<Classe>** no visitante:

```
def accept(self, visitor):  
    method = 'visit_' + type(self).__name__  
    visit = getattr(visitor, method, visitor.generic_visit)  
    return visit(self)
```

Os principais nós da AST são:

Nó	Campos	Descrição
Program	units	Raiz; lista de unidades de programa
MainProgram	name, body	Programa principal
FunctionDef	name, params, return_type, body	Função com ou sem tipo explícito
SubroutineDef	name, params, body	Subrotina

Nó	Campos	Descrição
Body	<code>declarations, statements</code>	Corpo de uma unidade
TypeDecl	<code>type_name, char_len, variables</code>	Declaração de tipo
Assign	<code>target, value</code>	Atribuição
IfThen	<code>condition, then_body, else_body</code>	IF-THEN-[ELSE]-ENDIF
LogicalIf	<code>condition, stmt</code>	IF lógico de linha única
DoLoop	<code>label, var, start, stop, step, body</code>	Ciclo DO
Goto	<code>label</code>	GOTO
CallStmt	<code>name, args</code>	CALL sub
VarOrFuncCall	<code>name, args</code>	Array <code>A(i)</code> ou chamada <code>F(x)</code> (ambíguo)
FuncCall	<code>name, args</code>	Função intrínseca
ArithmeticBinOp	<code>left, op, right</code>	Operação aritmética binária
RelationalBinOp	<code>left, op, right</code>	Comparação
LogicalBinOp / LogicalUnaryOp	...	Lógica booleana

Uma decisão de design relevante foi o nó `VarOrFuncCall`: a sintaxe `NAME(args)` é sintaticamente ambígua — pode ser acesso a array ou chamada de função de utilizador. Como a distinção requer a tabela de símbolos, o parser cria sempre `VarOrFuncCall` e delega a resolução para a análise semântica. As funções intrínsecas (`MOD`, `SQRT`, `MAX`, `MIN`), por terem tokens próprios e produções dedicadas, são sempre representadas por `FuncCall`.

A recuperação de erros usa **panic mode por linha**: ao encontrar um token inesperado, o parser descarta tokens até ao próximo `NEWLINE` e chama `errok()`, permitindo reportar múltiplos erros numa só passagem.

Análise Semântica

O `SemanticAnalyser` herda de `ASTVisitor` e percorre a AST verificando a correção semântica antes da tradução. Acumula todos os erros encontrados sem parar no primeiro, permitindo ao utilizador ver todos os problemas de uma vez.

Tabela de Símbolos

A `SymbolTable` é uma lista de dicionários (um por scope), onde `_table[0]` é sempre o scope global. Suporta dois tipos de entradas:

- **VarSymbol**: guarda nome, tipo (**INTEGER**, **REAL**, **LOGICAL**, **CHARACTER** ou **UNKNOWN**), dimensão (0 = escalar, ≥ 1 = array), flag **initialized**, e o índice no frame da VM. Cada array ocupa apenas 1 slot no frame, pois esse slot guarda o endereço heap retornado por **ALLOC**.
- **SubprogramSymbol**: guarda nome, kind (**FUNCTION**/**SUBROUTINE**), tipo de retorno, e listas de nomes e tipos dos parâmetros.

Os parâmetros formais recebem índices **negativos** (parâmetro 0 → índice -1, parâmetro 1 → índice -2, ...), refletindo diretamente o layout do stack frame da EWVM onde **fp[-1]** é o primeiro argumento. As variáveis locais têm índices não-negativos a partir de 0. Nas funções, a variável de retorno (com o mesmo nome da função) ocupa sempre **fp[0]**.

Fases da análise

Fase 1 — Pré-registo de subprogramas: antes de analisar qualquer unidade, todos os subprogramas são registados no scope global. Os tipos dos parâmetros e o tipo de retorno são inferidos das declarações presentes no body sem criar scope. Isto permite chamadas diretas a subprogramas definidos mais à frente no ficheiro.

Fase 2 — Análise por unidade: para cada unidade é criado um novo scope. Os parâmetros são declarados primeiro (com índices negativos), seguidos das declarações de tipo do body — se uma variável já existe com tipo **UNKNOWN** (parâmetro sem tipo), o tipo é refinado sem alterar o índice. As instruções são então verificadas.

Fase 3 — Validação de labels: no final de cada unidade, **_check_label_refs()** verifica que todos os labels referenciados por **GOTO** e **DO** estão definidos nessa unidade. Labels são locais a cada unidade de programa.

Verificações implementadas

Verificação	Descrição
Re-declaração de variáveis	Erro se a variável já estiver declarada no scope atual com tipo conhecido
Compatibilidade de tipos	INTEGER ↔ REAL permitida; outros cruzamentos dão erro
Condição do IF	Deve ser do tipo LOGICAL
Variável de controlo DO	Deve ser INTEGER ou REAL ; não pode ser modificada dentro do body
Limites e passo do DO	Devem ser numéricos
Acesso a array	Exactamente 1 índice; o índice deve ser INTEGER
Subrotina com CALL	Aridade correta; tipos dos argumentos verificados se conhecidos
FUNCTION com CALL	Erro — funções não se chamam com CALL
Subrotina como expressão	Erro — subrotinas não produzem valor
Inicialização da função	Avisa se a variável de retorno pode não ter sido atribuída
Labels duplicados	Erro se o mesmo label for definido mais de uma vez na mesma unidade
Labels indefinidos	Erro para cada GOTO/DO que referencia um label não definido

Inferência de tipos

O método `_type_of(node)` percorre a expressão recursivamente e devolve o tipo como string ou `None` se indeterminável:

Expressão	Tipo devolvido
<code>IntLit</code> , <code>RealLit</code> , <code>StringLit</code> , <code>BoolLit</code>	<code>INTEGER</code> , <code>REAL</code> , <code>CHARACTER</code> , <code>LOGICAL</code>
<code>Var</code>	tipo da symbol table
Operação aritmética mista (<code>INTEGER op REAL</code>)	<code>REAL</code> (promoção numérica)
Operação aritmética homogênea	tipo dos operandos
Operação relacional	sempre <code>LOGICAL</code>
Operação lógica	sempre <code>LOGICAL</code>
<code>MOD(a, b)</code>	<code>INTEGER</code>
<code>SQRT(x)</code>	<code>REAL</code>
<code>MAX/MIN</code> com algum <code>REAL</code>	<code>REAL</code> ; caso contrário <code>INTEGER</code>

Tradução

A fase de tradução é composta por três módulos orquestrados pelo **Translator**: **IRBuilder** (AST → IR), **IROptimizer** (IR → IR otimizada) e **IRCodeGen** (IR → código EWVM).

Representação Intermédia (IR)

A IR é uma sequência plana de instruções tipadas (`IRInstr`), organizada em unidades (`IRUnit`) correspondentes a cada `PROGRAM`, `FUNCTION` ou `SUBROUTINE`. A introdução desta camada entre a AST e o output textual traz várias vantagens: separa a lógica de geração de código da lógica de otimização e do formato de output; permite otimizações locais sobre sequências lineares de instruções, mais simples do que sobre a AST. As principais categorias de instruções IR são:

Categoria	Instruções IR (→ comandos EWVM)
Literais	<code>IRPushI</code> → <code>PUSHI</code> , <code>IRPushF</code> → <code>PUSHF</code> , <code>IRPushS</code> → <code>PUSHS</code> , <code>IRPushN</code> → <code>PUSHN</code>
Variáveis (frame)	<code>IRPushG/IRStoreG</code> → <code>PUSHG/STOREG</code> , <code>IRPushL/IRStoreL</code> → <code>PUSHL/STOREL</code>
Heap (arrays)	<code>IRAlloc</code> → <code>ALLOC</code> , <code>IRLoadN</code> → <code>LOADN</code> , <code>IRStoreN</code> → <code>STOREN</code>
Aritmética inteira	<code>IRAdd</code> , <code>IRSub</code> , <code>IRMul</code> , <code>IRDiv</code> , <code>IRMod</code> → <code>ADD</code> , <code>SUB</code> , <code>MUL</code> , <code>DIV</code> , <code>MOD</code>
Aritmética real	<code>IRFAdd</code> , <code>IRFSub</code> , <code>IRFMul</code> , <code>IRFDiv</code> → <code>FADD</code> , <code>FSUB</code> , <code>FMUL</code> , <code>FDIV</code>
Comparações int/real	<code>IRInf/IRFInf</code> , <code>IRInfEq/IRFInfEq</code> , <code>IRSup/IRFSup</code> , <code>IRSupEq/IRFSupEq</code> , <code>IREqual</code>
Lógica	<code>IRAnd</code> → <code>AND</code> , <code>IROr</code> → <code>OR</code> , <code>IRNot</code> → <code>NOT</code>

Categoria	Instruções IR (→ comandos EWVM)
Conversões	<code>IRItOf</code> → <code>ITOf</code> , <code>IRFtoI</code> → <code>FTOI</code> , <code>IRAtOI</code> → <code>ATOI</code> , <code>IRAtOf</code> → <code>ATOf</code>
Controlo de fluxo	<code>IRLabel</code> , <code>IRJump</code> → <code>JUMP</code> , <code>IRJZ</code> → <code>JZ</code> , <code>IRPushA</code> → <code>PUSHA</code> , <code>IRCall</code> → <code>CALL</code> , <code>IRReturn</code> → <code>RETURN</code>
I/O	<code>IRRead</code> → <code>READ</code> , <code>IRWriteI/IRWriteF/IRWriteS</code> → <code>WRITEI/WRITEF/WRITES</code> , <code>IRWriteLn</code> → <code>WRITELN</code>
Pilha	<code>IRPop</code> → <code>POP</code> , <code>IRSwap</code> → <code>SWAP</code> , <code>IRCopyK</code> → <code>COPY</code>

A operação `.NE` não tem comando direto na EWVM e é emitida como `EQUAL` seguido de `NOT`.

Geração de código (`IRBuilder`)

O `IRBuilder` percorre a AST com o padrão Visitor e emite instruções IR para a `IRUnit` ativa. O layout do stack frame segue a convenção da EWVM: variáveis globais em `gp[0..N]`, variáveis locais em `fp[0..M]`, parâmetros em `fp[-1]`, `fp[-2]`,

Para funções, o caller (chamador) reserva um slot de retorno antes de empilhar os argumentos em ordem inversa (o último argumento fica mais fundo, o primeiro fica em `fp[-1]`):

```
PUSHI 0 ; slot de retorno (será preenchido pela função)
<eval argN> ... <eval arg1> ; argumentos em ordem inversa
PUSHA nome_funcao
CALL
POP arity ; caller limpa os argumentos após RETURN
; slot de retorno ficou no topo
```

Antes do corpo de cada função/subrotina, são alocados todos os slots locais com `PUSHN n` e de seguida emitidos `ALLOC+STORE` para cada array local. Nas funções, o valor de retorno é copiado para o slot reservado pelo caller (`STOREL -(arity+1)`) antes de limpar os locais e emitir `RETURN`.

Arrays são alocados na heap com `ALLOC n`, guardando apenas o endereço no frame. O acesso a elementos usa `LOADN/STOREN` com conversão de índice Fortran 1-based para 0-based:

```
PUSHG/PUSHL addr_idx ; endereço da heap (slot do array no frame)
<eval índice>
PUSHI 1
SUB ; converte para 0-based
<eval valor> ; (só na escrita)
LOADN / STOREN
```

O código gerado para estruturas de controlo segue padrões fixos. Para o IF-THEN-ELSE:

```
<eval condição>
JZ ELSElabelN
```

```

<then_body>
JUMP ENDIFlabelN
ELSElabelN:
<else_body>
ENDIFlabelN:

```

Para o DO loop, a variável de controlo é inicializada e a condição `var <= stop` é verificada no início de cada iteração:

```

<eval start>; STOREG/STOREL var
D0labelN:
  PUSHG/PUSHL var; <eval stop>; INFEQ
  JZ ENDD0labelN
  <body>
  PUSHG/PUSHL var; <eval step | PUSHI 1>; ADD; STOREG/STOREL var
  JUMP D0labelN
ENDD0labelN:

```

O operador `**` não existe na EWVM e é implementado como chamada a uma função auxiliar `POWERFUNClabel`, gerada condicionalmente no final do output apenas quando usada. Esta função segue a mesma convenção de chamada das funções de utilizador (arity=2: base em `fp[-1]`, expoente em `fp[-2]`) e calcula a potência por multiplicações sucessivas num ciclo inteiro.

Otimizações (IROptimizer)

O otimizador aplica quatro passagens em ciclo até convergência (máximo 20 iterações de segurança):

1. **Constant folding:** sequências `PUSHI/PUSHF + PUSHI/PUSHF + operação binária` são substituídas pela constante resultante. Cobre operações inteiras e reais, com promoção automática inteiro → real.
2. **Eliminação de código morto:** remove instruções inacessíveis após `JUMP/RETURN/STOP` até ao próximo label.
3. **Eliminação de saltos triviais:** `JUMP label` imediatamente seguido de `label:` é eliminado.
4. **Eliminação de labels órfãos:** remove labels não referenciados por nenhum `JUMP`, `JZ` ou `PUSHA`. Labels de subprogramas são protegidos por um conjunto `global_pusha` calculado antes da otimização.

Dificuldades encontradas

Uma das dificuldades mais significativas foi a resolução da ambiguidade do DO loop no autómato LALR(1). A gramática do Fortran 77 exige que o loop DO termine com um `LABEL CONTINUE` específico, mas o parser não conseguia distinguir, dentro do `stmt_section` do body, se esse par pertencia ao body ou era o terminador do loop. A solução passou pela introdução do pseudotoken `DO_TERM` com a maior precedência na tabela PLY, forçando a redução correta da produção do DO antes de o parser tentar consumir o `LABEL CONTINUE` como instrução interior.

Outra dificuldade foi a ambiguidade sintática de `NAME(args)`, que em Fortran 77 pode ser tanto um acesso a elemento de array como uma chamada a função definida pelo utilizador. Como o parser não tem acesso à tabela de símbolos, foi necessário criar o nó `VarOrFuncCall` para representar ambos os casos e delegar a desambiguação para a análise semântica. Esta fase resolve o caso com base na tabela de símbolos: se `NAME` estiver declarado como array, é acesso; se estiver registado como função, é chamada; se nenhum dos dois, é erro. Esta abordagem implicou um cuidado adicional no `IRBuilder`, que tem de realizar o mesmo lookup para emitir o código correto.

Por fim, as limitações da EWVM apresentaram desafios na geração de código para algumas funcionalidades do Fortran 77. A ausência de instrução nativa de potência levou à implementação da função auxiliar `POWERFUNClabel`, que segue a mesma convenção de chamada das funções de utilizador e é emitida condicionalmente no final do output. A ausência de instrução de raiz quadrada (`SQRT`) não pôde ser contornada de forma equivalente e resulta em terminação do programa com mensagem de erro em runtime.

Intruções de utilização do compilador

Para correr e testar o compilador (com todas as análises e tradução), basta correr os seguintes comandos:

```
cd fortran77-compiler/  
python __main__.py <path para o ficheiro Fortran 77>
```

Para mais informações sobre a execução passo a passo do compilador, basta correr:

```
python __main__.py -h  
  
usage: python __main__.py [-h] [-pp | -l | -p | -s | -t] file  
  
Compilador de Fortran 77  
  
positional arguments:  
  file                Path para o ficheiro Fortran 77  
  
options:  
  -h, --help          show this help message and exit  
  -pp, --preprocess    Executa apenas o pré-processamento  
  -l, --lexer          Executa o lexer  
  -p, --parser         Executa o lexer e o parser  
  -s, --semantic       Executa o lexer, parser e a análise semântica  
  -t, --translate      Executa a tradução completa
```

Os ficheiros de output com o código traduzido para linguagem da máquina EWVM serão guardados na pasta `fortran77-compiler/test/vm/`.

Para mais informações sobre a execução do compilador, consulte o ficheiro [README.md](#).

Conclusão

Com o desenvolvimento deste projeto, fomos capazes de abordar os vários conceitos e técnicas, resultando num compilador básico e funcional para a linguagem Fortran 77 Standard.

O processo de construção do compilador permitiu-nos compreender melhor as diversas etapas do *pipeline* do processamento de linguagens, desde a análise léxica até à geração de código. A utilização da biblioteca *PLY* facilitou a implementação das análises léxica e sintática, tornando o processo mais eficiente e organizado.

A construção deste compilador não só reforçou os conhecimentos teóricos adquiridos ao longo do semestre, mas também nos proporcionou uma experiência prática valiosa na área de processamento de linguagens e compiladores.

Ficheiros de suporte

- [Gramática](#)
- [Documentação da linguagem da máquina EWVM](#)

Bibliografia

[1] D. M. Beazley, "PLY (Python Lex-Yacc) — Documentation", disponível em:
<https://www.dabeaz.com/ply/ply.html>

[2] Oracle Corporation, "Fortran 77 Reference Manual", Sun Microsystems, disponível em:
<https://docs.oracle.com/cd/E19957-01/805-4939/index.html>